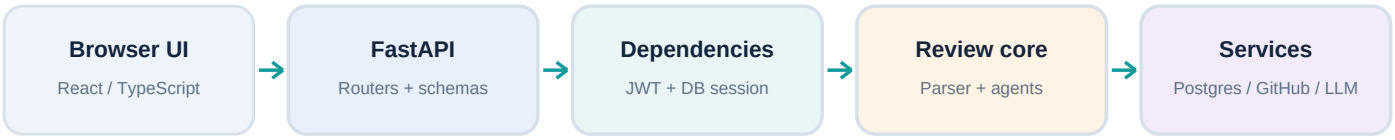
FastAPI Architecture

How the Multi-Agent Code Review System moves from a browser request to a persisted, structured review

Project snapshot

The web backend is an API layer around the existing code-review engine. FastAPI owns application setup, routing, request parsing, dependency injection, HTTP responses, and streaming. The review engine owns diff parsing and specialist analysis. SQLAlchemy stores users, reviews, findings, conversations, and configuration; provider and GitHub clients connect the application to external services.



A request moves through these layers; the database and external providers are reached by the relevant route/core service.

The central design idea

Routes are grouped by feature and mounted under **/api**. A route accepts a Pydantic request model, receives shared resources such as a database session or current user through **Depends(...)**, calls a domain service/orchestrator, and returns a typed response model. That keeps HTTP details at the edge while the parser, agents, and result models can also be used from the CLI and tests.

This guide describes the current checked-in implementation. It distinguishes behavior implemented in code from the broader product description in the milestone notes.

Quick mental model

Layer	Main responsibility	Project location
Web app	Create FastAPI app, middleware, startup, route mounting, health/static serving	src/code_review/web/app.py
HTTP features	Auth, reviews, chat, GitHub, PRs, agents, providers, profile, metrics	src/code_review/web/routes/*.py
Core review	Parse unified diff; run specialist agents; merge and rank findings	parsers/, agents/, orchestrator.py
Persistence and providers	SQLAlchemy records/sessions; Ollama and NVIDIA clients	db/, llm/, github/

1. Application setup and routing

The ASGI application object is exported as **code_review.web.app:app**. The module calls **create_app()** once at import time, while tests and benchmarks can call the factory to create a fresh app instance.

Startup lifecycle

FastAPI's lifespan context runs at startup. It calls **init_database()** to create any missing SQLAlchemy tables, then opens a session and seeds default records such as agent configuration. The engine and session factory are cached. Each route that requests **get_db** gets a session which is closed in the dependency's **finally** block after the request.

The startup handler logs and catches initialization exceptions, then continues to yield. This means a database initialization problem is logged but does not itself abort application startup; requests requiring the database can fail later.

Middleware and static content

- CORS allows the local Vite/React origins (ports 5173/3000) and local backend origins, plus the configured frontend URL.
- The **/avatars** mount serves avatar files from **frontend/public/avatars**.
- If **frontend/dist** exists, built assets are served under **/assets** and a catch-all route returns the SPA index for browser navigation.
- **/api/health** returns status, app name, version, and the configured active LLM provider.

Feature routers

Router	Representative paths	Purpose
Auth	/api/auth/login, /api/auth/signup, /api/auth/me, /api/auth/google/*	Password/JWT and Google OAuth entry points
Reviews	/api/reviews and /api/reviews/{id}	Start review, browse, inspect, delete history
Conversations	/api/conversations/*	Persisted assistant threads, messages, feedback, SSE
GitHub / PRs	/api/github/*, /api/pull-requests/*	OAuth connection, repos, diffs, PR metadata/review
Agents / providers	/api/agents, /api/providers/*	Agent config and LLM provider status/test
Overview / user	/api/overview/metrics, /api/users/*	Dashboard metrics and profile/preferences

Most routers are mounted with a shared **/api** prefix, while each router declares its own feature prefix. The auth router is also mounted without **/api** for the configured Google callback URI. User routes have both **/api/user** and **/api/users** prefixes for compatibility.

2. Request handling: from JSON to response

A typical JSON endpoint follows this sequence:

1. **Route match**

   Starlette/FastAPI matches the HTTP method and path to a function registered on an APIRouter.

2. **Parse + validate**

   Pydantic request schemas validate JSON types, required fields, formats, and bounds before route logic runs.

3. **Inject dependencies**

   Depends(...) resolves a SQLAlchemy session and, when requested, a user from the Bearer token or session cookie.

4. **Execute feature logic**

   The route queries or updates SQLAlchemy models and calls a focused service or the review orchestrator.

5. **Serialize response**

   Response models convert ORM objects/domain values into a stable JSON shape and filter fields to the declared schema.

3. Main code-review lifecycle

The principal endpoint is **POST /api/reviews**. Its body uses **ReviewCreateRequest**, which includes diff text, title/repository/PR labels, a provider choice, an include-LLM flag, and an optional agents list.

Stage	What happens in this project
Request	The route reads the payload, selects a provider/model, checks enabled agent config, and starts a wall-clock timer.
Configure	It creates ReviewOrchestrator with parallel execution enabled. LLMAgent is included only when requested and the enabled config set contains one of the recognized LLM-related IDs.
Parse	review_diff() calls parse_diff() to turn unified diff text into DiffFile objects: changed paths, hunks, line types, and added-line numbers.
Analyze	ASTAgent, RuffAgent, and BanditAgent perform deterministic checks. When enabled, LLMAgent sends per-file added-line context to the selected model.
Combine	The orchestrator collects each agent's findings, tolerates individual agent exceptions, deduplicates by file/line/rule-or-message, and sorts by severity then path/line.
Persist	The route creates a ReviewRecord and linked FindingRecord rows, commits them, refreshes the review, and returns ReviewDetailResponse.

Result shape

A review response contains identifying metadata (repository, PR identifier, provider/model, duration, agent names and finding counts), the original diff, and a list of findings. Each finding includes agent, title/description, severity/category, path and line, optional suggested fix, rule ID, and evidence.

4. Review engine and AI providers

The HTTP backend does not contain the analysis rules itself. It hands the parsed diff to an orchestrator and receives typed domain objects back.

Agent responsibilities

Agent	Role	Typical input behavior
ASTAgent	Syntax / structural checks	Works from parsed changed files and reports structured findings.
RuffAgent	Python lint and quality rules	Invokes Ruff for relevant Python files and maps diagnostics into findings.
BanditAgent	Python security patterns	Runs Bandit on Python files and filters results to changed lines.
LLMAgent	Semantic/contextual review	Checks provider availability; prompts for each changed file with additions; validates returned findings against added lines.

Parallelism and failure handling

The web route asks the orchestrator to run agents in parallel. The orchestrator uses **ThreadPoolExecutor**, waits for each result, and stores exceptions by agent name instead of aborting collection when one agent fails. It then optionally filters severity, deduplicates, sorts, and creates a **ReviewResult** with files reviewed and elapsed milliseconds. This is a separate layer of concurrency from handling multiple HTTP requests.

LLM provider path

get_llm_client() is a small factory. It returns either **OllamaClient** (local HTTP endpoint, default model configured in settings) or **NemotronClient** (NVIDIA NIM using an API key). Both satisfy a protocol with availability, regular chat, and streaming chat methods. The same factory supports semantic reviews and the assistant conversation endpoints. Provider status/test routes probe availability and report models/latency.

Chat streaming

Conversation routes support ordinary JSON endpoints and streaming endpoints. Streaming returns a **text/event-stream** response with named SSE events: **init** for new-thread metadata, **token** for each generated chunk, **done** with the assembled answer and timing metrics, and **error** with a normalized user-facing error. Generation history is limited to the most recent eight messages; optional diff context is clipped to 8,000 characters. The assistant reply is saved with a separate DB session after generation (partial output is also preserved if streaming fails after chunks were sent).

The UI's **frontend/src/services/api.ts** wraps fetch calls, attaches the Bearer token when present, includes cookies, parses SSE events, and maps API errors to user-facing text.

Performance and operational implication

Review execution is synchronous from the client's perspective: **POST /api/reviews** returns after analysis and database commit. The route is declared with plain **def**, and the selected agent work may itself wait on subprocesses or remote model calls. Assistant chat has SSE specifically so the browser can display incremental output. There is no task queue or background-job handoff in this path.

5. Authentication, data, and integrations

Authentication model

Signup/login hash passwords with bcrypt, create a signed JWT using HS256, and place the token in a session cookie while also returning it in the response body. The token lifetime is seven days. The auth dependency accepts an HTTP Bearer token first, then falls back to the **nexarion_session** cookie. **get_current_user** raises HTTP 401 if no valid user resolves; **get_current_user_optional** returns **None** for anonymous requests.

Routes choose whether authentication is required. The pull-request listing and overview metrics require a user. Reviews and conversations use optional authentication: records are associated with the signed-in user when present, otherwise with a null user ID. Review history list returns an empty list for anonymous users; detail/delete queries scope to either that user's records or anonymous records.

Persistence model

Tables	Relationships / purpose
users	Identity, preferences, GitHub connection fields; one user can own reviews.
reviews -> findings	A review stores the full diff and run metadata; findings are child rows linked by review_id.
agent_configs	Seeded agent definitions and enabled/provider/model/options values.
conversations → messages	Conversation threads own ordered messages; messages store role, content, feedback, and time.
repositories / pull_requests	Repository and PR record types are available alongside live GitHub-backed API paths.

SQLAlchemy uses a cached Engine and sessionmaker. **get_db** is a yield dependency that guarantees session closure. The models use JSON with a PostgreSQL JSONB variant for list/dict fields such as agent IDs, capabilities, and options.

External services

- GitHub integration uses a service class with a per-user GitHub token for accessible repositories, PRs, and diffs. OAuth routes handle connect/callback/disconnect.
- Google OAuth endpoints generate an authorization URL and process the callback. A root-level auth router mount supports the configured callback URI without the API prefix.
- Cloudinary settings support avatar storage; user profile routes handle avatar upload and related profile state.
- Ollama is a local model service; NVIDIA NIM is a cloud service. Provider configuration is loaded through Pydantic Settings from environment variables and an optional .env file.

6. Current implementation notes and code map

The following details are useful when reading the code or extending the API. They describe the current implementation, including places where configuration surfaces are broader than the execution behavior.

Area	Current behavior / implication
Agent selection	The request schema has an agents list, but create_review does not consume it. Enabled agent configs are used to decide whether to include the LLM; the default AST, Ruff, and Bandit agents are still constructed together.
Agent config scope	The current /api/agents handlers query/update by agent_id and do not apply a user_id filter, so configurations behave like shared system-level settings in these endpoints.
Provider defaults	ReviewCreateRequest defaults provider to 'nvidia'; chat defaults through Settings (which currently defaults llm_provider to 'ollama'). The review route uses the request value when it is one of the supported providers.
Review error response	If orchestration raises, the route returns HTTP 500 with the exception string in detail. AI conversation errors go through a normalizer that returns friendlier messages.
Startup DB failure	Lifespan logs initialization/seed errors and continues startup. Deployment health only reports app/provider fields, not database readiness.
Packaging	The current pyproject.toml declares the early/core dependencies but does not list the FastAPI, SQLAlchemy, DB driver, or auth packages imported by the web implementation. A fresh install therefore needs the web runtime dependencies supplied separately.

Where to read next

Question	Primary files
How is the web app assembled?	src/code_review/web/app.py; src/code_review/web/__init__.py
What does the review endpoint do?	src/code_review/web/routes/review_routes.py; src/code_review/web/schemas.py
How are agents coordinated?	src/code_review/orchestrator.py; src/code_review/agents/*.py; src/code_review/parsers/diff_parser.py
How are auth and DB sessions resolved?	src/code_review/web/auth.py; src/code_review/db/database.py; src/code_review/db/models.py
How does chat streaming work?	src/code_review/web/routes/conversation_routes.py; src/code_review/llm/factory.py; frontend/src/services/api.ts
What config controls services?	src/code_review/config.py; .env.example

In one sentence

FastAPI is the project's HTTP composition layer: it validates browser input, injects shared state, delegates work to the review/chat domain components, and serializes results while SQLAlchemy and provider clients handle persistence and integrations.

Prepared from source inspection of the repository on 27 October 2026. No application code was changed for this guide.